TRESORIA

Logiciel de gestion financière

MASTER PROJECT BACKUP / HANDOFF

Document de continuité

ÉTAT CONSOLIDÉ DU PROJET

Architecture, métier, sécurité, décisions, historique, déploiement, UAT et règles de reprise.

But de ce document : permettre à un futur développeur, une future IA ou toi-même de reprendre le projet avec le minimum de dépendance aux anciennes conversations.

Important : ce document distingue explicitement ce qui est vérifié, ce qui est décidé mais pas encore vérifié dans l'état final, et ce qui est à confirmer. Je n'ajoute pas de technologie, de fonctionnalité ou de décision qui n'est pas soutenue par l'historique disponible.

CLIENT DE RÉFÉRENCE

EL-BARKA EMBALLAGE

Document conçu pour la continuité technique du projet Tresoria.

TABLE DES MATIÈRES

Navigation structurelle du dossier

01 PROJECT IDENTITY

02 ORIGINAL IDEA AND PROJECT EVOLUTION

03 CURRENT APPLICATION ARCHITECTURE

04 COMPLETE FILE AND FOLDER STRUCTURE

05 TECHNOLOGY STACK

06 DEPENDENCIES

07 FRONTEND

08 BACKEND

09 DATABASE / STORAGE

10 AUTHENTICATION AND ADMIN SYSTEM

11 ENVIRONMENT VARIABLES

12 SECURITY

13 EMAIL / CONTACT SYSTEM

14 REVIEWS / RATINGS SYSTEM

15 ANALYTICS

16 IMPORTANT UI/UX DETAILS

17 DEPLOYMENT

18 LOCAL DEVELOPMENT

19 BUGS AND PROBLEMS ENCOUNTERED

20 COMPLETED FEATURES

21 PARTIALLY COMPLETED FEATURES

22 NOT IMPLEMENTED / FUTURE IDEAS

23 CURRENT KNOWN ISSUES

24 IMPORTANT DECISIONS

25 THINGS FUTURE AI/DEVELOPERS MUST NOT CHANGE

26 HOW TO REBUILD THE PROJECT FROM ZERO

27 CURRENT PROJECT STATE SNAPSHOT

28 NEXT DEVELOPMENT STEPS

29 RECOVERY CHECKLIST

30 INFORMATION THAT COULD NOT BE RECOVERED

31 FILES I MUST KEEP OUTSIDE CHATGPT

32 MASTER RESUME PROMPT

APPENDIX A — ENGINEERING WORKFLOW RECOVERED

APPENDIX B — QUALITY GATES

APPENDIX C — UAT DATASET KNOWN

APPENDIX D — FINAL HANDOFF RULE

TRESORIA / PROJECT KNOWLEDGE BASE

1. PROJECT IDENTITY

1.1 Nom du projet

Tresoria

Nom retenu pour le logiciel de gestion financière.

Le projet a initialement été désigné comme :

Logiciel de gestion financière

L'identité commerciale discutée ensuite est :

> **Tresoria**

Le client de référence est :

> **EL-BARKA EMBALLAGE**

1.2 Nature du logiciel

Tresoria est une **application desktop Windows locale**, destinée à la gestion financière opérationnelle d'une entreprise.

Elle fonctionne localement et hors ligne.

Elle n'est pas conçue comme :

- un SaaS ;
- une application web ;
- un ERP complet ;
- un logiciel bancaire ;
- une plateforme comptable complète ;
- un système cloud obligatoire ;
- une architecture microservices.

L'architecture retenue est volontairement un **monolithe modulaire**, afin de rester simple, maintenable et adapté à la taille du projet.

1.3 Objectif principal

Le logiciel doit permettre de suivre notamment :

- les ventes ;
- les achats ;
- les encaissements clients ;
- les paiements fournisseurs ;
- les créances clients ;
- les dettes fournisseurs ;
- les dépenses ;

- la caisse ;
- la banque ;
- les transferts internes ;
- les clients ;
- les fournisseurs ;
- les utilisateurs ;
- les permissions ;
- les alertes ;
- le Dashboard ;
- les rapports financiers ;
- les sauvegardes/restaurations.

Le principe métier fondamental est que **les flux financiers doivent être fiables et cohérents**, particulièrement lorsqu'une opération est composée de plusieurs écritures.

TRESORIA / PROJECT KNOWLEDGE BASE

2. ORIGINAL IDEA AND PROJECT EVOLUTION

2.1 Situation initiale

Le client disposait déjà d'un autre logiciel pour certaines opérations commerciales, notamment autour du :

- stock ;
- produits ;
- ventes opérationnelles ;
- fournisseurs ;
- tickets/factures.

Tresoria a donc été pensé comme une application **indépendante**.

Il n'est pas prévu de partager la base de données avec cet autre logiciel.

Il n'y a pas d'intégration automatique entre les deux systèmes dans le V1 actuel.

2.2 Compréhension initiale du besoin

Le besoin a progressivement été clarifié autour de :

```
VENTES
ACHATS
  ↓
OBLIGATIONS
  ↓
PAIEMENTS
  ↓
TRÉSORERIE
```

avec en parallèle :

```
DÉPENSES
CLIENTS
FOURNISSEURS
UTILISATEURS
```

Puis :

```
DASHBOARD
RAPPORTS
SAUVEGARDE / RESTAURATION
SÉCURITÉ
```

2.3 Évolution importante du modèle financier

Une décision fondamentale a été prise :

Les paiements ne sont pas simplement des lignes indépendantes.

Une vente constitue une **obligation**.

Un achat constitue également une **obligation**.

Les paiements peuvent être partiels et multiples.

Exemple :

```
Vente = 3 000

Paiement 1 = 500
Paiement 2 = 1 000
Paiement 3 = 1 200
Paiement 4 = 300

Total payé = 3 000
Reste = 0
```

Pour les paiements, l'allocation doit être explicite.

Un paiement n'est **jamais automatiquement affecté** à une dette.

2.4 Évolution du Dashboard

Le Dashboard avait initialement été envisagé avec une logique de comparaison.

Cette décision a été modifiée.

Décision finale

Dashboard = vue opérationnelle d'une période.

Rapports = analyse, comparaison et évolution.

Dashboard :

- Aujourd'hui ;
- Ce mois ;
- Cette année ;
- Personnalisé.

Il n'y a **pas de preset “Cette semaine”** dans la conception retenue.

Les comparaisons ont été déplacées vers le module **Rapports**.

2.5 Évolution de l'UX des paiements

Le client a demandé de ne plus présenter les paiements comme une longue liste de petits paiements.

La vue principale doit être **centrée sur l'obligation**.

Encaissements clients

Une ligne principale = une vente/créance.

Elle doit afficher notamment :

- client ;
- référence vente ;

- montant initial ;
- total payé ;
- reste ;
- statut ;
- action.

Puis :

Détails

ouvre l'historique des paiements.

Exemple :

```
Vente : 3 000
Payé : 2 700
Reste : 300
```

Détails :

```
500  → reste 2 500
1 000 → reste 1 500
1 200 → reste 300
```

Cette logique a été explicitement demandée dans les documents métier accessibles.

Même principe pour les fournisseurs :

```
1 ligne = 1 achat / dette
```

TRESORIA / PROJECT KNOWLEDGE BASE

3. CURRENT APPLICATION ARCHITECTURE

3.1 Architecture générale

Architecture retenue :

```
FinancialManagement.Domain
        ↑
FinancialManagement.Application
        ↑
FinancialManagement.Infrastructure
        ↑
FinancialManagement.Desktop
```

Avec les tests séparés.

3.2 Philosophie architecturale

Le projet utilise un :

> **Monolithe modulaire**

et non :

- microservices ;
- Docker ;
- CQRS complexe ;
- event sourcing ;
- backend distant obligatoire.

Cette décision était volontaire : le projet est local, mono-poste et doit rester simple.

3.3 Domain

Responsabilité :

- règles métier ;
- entités ;
- value objects/domain concepts ;
- invariants.

Le Domain ne doit pas dépendre de la couche WPF.

3.4 Application

Responsabilité :

- cas d'utilisation ;
- services applicatifs ;

- autorisation applicative ;
- orchestration des opérations ;
- interfaces nécessaires à l'infrastructure.

3.5 Infrastructure

Responsabilité :

- EF Core ;
- SQLite ;
- persistance ;
- migrations ;
- backup/restore ;
- services techniques ;
- accès aux données.

3.6 Desktop

Responsabilité :

- WPF ;
- MVVM ;
- Views ;
- ViewModels ;
- Shell ;
- navigation ;
- présentation ;
- interaction utilisateur.

3.7 Tests

Le projet possède une couverture automatisée importante.

Les validations réalisées au cours du projet ont atteint plusieurs checkpoints importants :

- 1109 PASS après Recovery V1 ;
- 1107/1107 sur une phase Dashboard/Reports ;
- 1111/1111 après correction finale des graphes/UI ;
- 1130 PASS lors de la validation combinée Recovery + Dashboard/Reports ;
- 1212 PASS au checkpoint Receipt V1.

Attention : ces nombres correspondent à différents états du projet et ne doivent pas être interprétés comme le nombre actuel de tests du HEAD final.

TRESORIA / PROJECT KNOWLEDGE BASE

4. COMPLETE FILE AND FOLDER STRUCTURE

4.1 Structure connue

La structure logique est :

```
FinancialManagement
│
├── FinancialManagement.Domain
│
├── FinancialManagement.Application
│
├── FinancialManagement.Infrastructure
│
├── FinancialManagement.Desktop
│
└── FinancialManagement.Tests
```

Le nom exact de la solution/repository Git n'est pas suffisamment documenté dans les éléments récupérés pour que je puisse garantir ici le nom exact du `.sln`.

4.2 Emplacements de développement connus

DEV1

```
C:\Dev\logiciel-gestion-financiere
```

DEV2

```
C:\Users\INPT\Dev\logiciel-gestion-financiere
```

Ces deux environnements étaient utilisés pour séparer les responsabilités.

4.3 Responsabilités DEV1

DEV1 :

- architecture ;
- backend ;
- base ;
- règles métier ;
- sécurité ;
- intégrité ;
- opérations financières ;
- migrations ;
- sauvegarde/restauration.

4.4 Responsabilités DEV2

DEV2 :

- WPF ;
- MVVM ;
- Shell ;
- navigation ;
- UI ;
- Dashboard ;
- Alertes ;
- composants visuels ;
- intégration UI.

4.5 Fichiers projet importants connus

Les noms exacts de tous les fichiers ne sont pas récupérables de manière fiable depuis l'historique, mais les composants suivants sont explicitement connus :

```
Application/
  CustomerPayments/
    CustomerPaymentReceipt.cs

Desktop/
  Services/
    CustomerPaymentReceiptDialogService.cs
    CustomerPaymentReceiptPrinter.cs

  Views/
    CustomerPaymentReceiptDocumentFactory.cs
    CustomerPaymentReceiptWindow.xaml
    CustomerPaymentReceiptWindow.xaml.cs
```

Tests :

```
CustomerPaymentReceiptServiceTests.cs
CustomerPaymentReceiptPresentationTests.cs
DueAlertsViewBindingTests.cs
```

4.6 Documents projet importants

Documents explicitement présents/accessibles dans l'historique :

```
Comprendre le métier client.txt

Sécuriser l’application installée.txt

Cahier des charges fonctionnel V1.0 — version validée
Modélisation fonctionnelle complète V1.0 — version validée
Conception du modèle de données V1.0 — version validée
Conception UX/UI V1.0 — version validée
Architecture Technique V1.0 — version validée
```

```
UAT-V1-COMPLETENESS-AUDIT.md

DASHBOARD-V2-STARTUP-DIAGNOSIS.md
DASHBOARD-V2-RUNTIME-CRASH-REPORT.md
DASHBOARD-V2-FINAL-VISUAL-QA-REPORT.md
DASHBOARD-V2-REPORT.md

fixture-manifest.committed.json
```

D'autres rapports Recovery/Astra/Payment/Receipt existaient, mais leurs noms exacts ne sont pas tous suffisamment certains pour être reconstruits ici.

TRESORIA / PROJECT KNOWLEDGE BASE

5. TECHNOLOGY STACK

Stack officielle connue

```
C#
.NET 10 LTS
WPF
MVVM
Entity Framework Core
SQLite
xUnit
Inno Setup
```

Cette stack est explicitement documentée dans la conception du projet.

Build

Le projet vise :

```
x64
self-contained
Release
```

Target Framework Desktop

Le Desktop a été adapté à :

```
net10.0-windows10.0.19041
```

afin de cibler Windows 10 tout en permettant l'utilisation sur Windows 11.

TRESORIA / PROJECT KNOWLEDGE BASE

6. DEPENDENCIES

Base technique

La dépendance ORM principale est :

```
Entity Framework Core
```

Base :

```
SQLite
```

Tests :

```
xUnit
```

UI :

```
WPF
```

Graphiques :

```
LiveChartsCore.SkiaSharpView.WPF
```

Version explicitement utilisée pour Dashboard/Reports :

```
2.0.5
```

OpenGL/graphique

Les problèmes de restauration de packages ont notamment concerné des dépendances WPF telles que :

```
SkiaSharp
OpenTK
```

Une validation d'intégration avait d'abord été partiellement bloquée par un timeout NuGet.

La revalidation ultérieure a réussi.

TRESORIA / PROJECT KNOWLEDGE BASE

7. FRONTEND

7.1 Framework

Frontend desktop :

```
WPF + MVVM
```

7.2 Shell

L'application possède un Shell principal avec navigation latérale.

Les modules connus comprennent notamment :

```
Dashboard
Ventes
Achats
Dépenses
Clients
Fournisseurs
Encaissements clients
Paiements fournisseurs
Trésorerie
Rapports
Administration
```

L'ordre exact de toutes les entrées peut avoir évolué ; **Rapports** a été placé avant **Administration**.

7.3 Design

L'objectif UI est :

- moderne ;
- professionnel ;
- lisible ;
- simple ;
- cohérent ;
- sans surcharge visuelle.

Les couleurs existantes doivent être conservées plutôt que d'ajouter gratuitement de nouvelles couleurs.

7.4 Dashboard

Le Dashboard V2 comprend :

- filtres de période ;
- KPI ;
- graphiques ;

- flux financiers ;
- positions financières courantes ;
- créances importantes ;
- dettes importantes ;
- alertes d'échéance.

Granularité des graphiques :

Aujourd'hui

Un point représentant la journée.

Pas de fausses données horaires.

Ce mois

Données quotidiennes.

Cette année

Données mensuelles :

```
Jan → Déc
```

Personnalisé

- ≤ 62 jours : quotidien ;
- > 62 jours : mensuel.

7.5 Rapports

Module réservé au Responsable.

Onglets :

```
Synthèse
Comparaison
Évolution
```

Synthèse

Analyse d'une période.

Comparaison

Comparaison :

```
Mois analysé
Mois de comparaison
```

avec bouton :

```
Comparer
```

Évolution

Une année avec évolution mensuelle :

```
Jan → Déc
```

7.6 Résultat financier

Le résultat opérationnel affiché est :

```
Ventes - Achats - Dépenses
```

Les paiements ne sont pas des revenus/dépenses supplémentaires.

Les transferts internes ne changent pas le résultat.

TRESORIA / PROJECT KNOWLEDGE BASE

8. BACKEND

8.1 Autorisation

Le backend doit toujours contrôler les droits.

Principe :

```
UI
 ↓
ViewModel
 ↓
Application Service
 ↓
Authorization
 ↓
Business Rule
 ↓
Database
```

Un bouton caché n'est pas considéré comme une protection suffisante. Le contrôle doit être réalisé côté backend/application.

8.2 Ventes

Une vente possède :

- référence automatique ;
- date ;
- montant ;
- client éventuel ;
- observation éventuelle ;
- paiements éventuels.

Les clients anonymes sont autorisés.

Mais une vente anonyme doit être entièrement payée.

Elle ne peut pas générer de crédit client.

8.3 Achats

Un achat possède :

- fournisseur obligatoire ;
- référence automatique ;
- date ;
- montant ;
- observation facultative ;
- paiements éventuels.

8.4 Paiements

Méthodes :

```
Cash
BankTransfer
```

Les paiements peuvent être :

- immédiats ;
- différés ;
- partiels ;
- multiples.

Ils sont explicitement affectés aux obligations.

8.5 Allocation

Règle fondamentale :

> Un paiement n'est jamais automatiquement affecté à une dette.

Exemple :

```
Paiement = 1 000

Vente A = 300
Vente B = 500

Allocation :
300 → A
500 → B

200 → avance
```

L'avance n'est pas stockée dans une table mutable de type :

```
AdvanceBalance
```

Elle est dérivée des paiements et allocations.

TRESORIA / PROJECT KNOWLEDGE BASE

9. DATABASE / STORAGE

9.1 SQLite

Base locale :

```
FinancialManagement.db
```

Emplacement de production connu :

```
C:\ProgramData\FinancialManagement\Data\FinancialManagement.db
```

9.2 Pourquoi ProgramData

Les données métier sont séparées des binaires de l'application.

Conceptuellement :

```
Program Files
   ↓
Application

ProgramData
   ↓
Données
   ↓
Backups
```

9.3 Montants

Décision critique :

Base

```
SQLite INTEGER
```

Domaine

```
long
```

Les montants persistés ne doivent pas utiliser :

```
float
double
```

Exemple :

```
10,50 €
```

est représenté en unités mineures.

Cette décision fait partie des choix structurants du modèle de données.

9.4 Migrations acceptées

Il existe exactement quatre migrations validées :

```
20260903232954_001_InitialSchema

20260904004853_002_AllowAnonymousSalePayments

20260904111121_003_TreasuryOpeningBalanceSingleton

20260908072131_004_EnforceFinancialIntegerStorageClass
```

Règle critique

Aucune migration 005 n'est acceptée dans le baseline connu.

9.5 Intégrité

Les opérations multi-écritures doivent être :

```
ALL
OR
NOTHING
```

Donc :

```
transaction
```

pour les opérations nécessitant plusieurs modifications cohérentes.

9.6 Trésorerie

V1 :

```
Cash
Bank
```

Les soldes sont dérivés.

Il n'y a pas de champ mutable de type :

```
CashBalance = ...
```

Les soldes négatifs sont autorisés.

9.7 Transferts

```
Cash → Bank
Bank → Cash
```

Un transfert :

- n'est pas une recette ;
- n'est pas une dépense ;
- ne modifie pas le résultat.

TRESORIA / PROJECT KNOWLEDGE BASE

10. AUTHENTICATION AND ADMIN SYSTEM

10.1 Utilisateurs

Deux niveaux :

```
RESPONSABLE
EMPLOYÉ
```

10.2 Responsable

Le Responsable possède les capacités sensibles, notamment :

- Dashboard global ;
- Trésorerie ;
- Administration ;
- Backup/Restore ;
- Rapports ;
- opérations administratives selon implémentation.

10.3 Employé

Les permissions sont configurables individuellement.

Modules :

```
Ventes
Achats
Dépenses
Clients
Fournisseurs
Encaissements clients
Paiements fournisseurs
```

Actions :

```
Voir
Créer
Modifier
```

10.4 B01 — lecture cross-user

Décision importante :

> Un employé ayant le droit de lecture peut voir les opérations créées par les autres utilisateurs.

Mais cela ne signifie pas qu'il peut librement les modifier.

Cette décision est volontairement conservée.

10.5 Session

Un problème de sécurité concernant les sessions après désactivation a été identifié dans un audit ultérieur :

```
Employé connecté
↓
Responsable désactive le compte
↓
ancienne session
↓
action encore possible
```

Ce problème avait été classé **SEC-001 — à corriger avant livraison** dans un état antérieur du projet.

État final de correction : à vérifier sur le HEAD actuel.

Ne pas considérer cette information comme définitivement résolue sans vérification Git/runtime.

10.6 Suppression

La suppression définitive des opérations financières était destinée au Responsable.

Cependant, les services complets de suppression permanente avaient été classés comme incomplets dans un audit de complétude.

Donc :

> **ne pas supposer que la suppression permanente complète est actuellement terminée.**

TRESORIA / PROJECT KNOWLEDGE BASE

11. ENVIRONMENT VARIABLES

Aucun inventaire fiable des **noms exacts de variables d'environnement** n'a été récupéré.

Par conséquent, ce document ne doit pas inventer :

```
DATABASE_PASSWORD
LICENSE_KEY
...
```

Les secrets ne doivent jamais être copiés dans ce document.

Configuration connue

Les informations structurelles connues sont :

```
Data directory:
C:\ProgramData\FinancialManagement\Data

Backup directory:
C:\ProgramData\FinancialManagement\Backups
```

Le système utilise SQLite localement.

TRESORIA / PROJECT KNOWLEDGE BASE

12. SECURITY

12.1 Modèle de menace

Le logiciel étant local/offline, la surface réseau est faible.

Les risques principaux sont :

1. accès physique au PC ;
2. accès Windows non autorisé ;
3. copie de l'application ;
4. copie de la base ;
5. copie des backups ;
6. modification des fichiers ;
7. reverse engineering ;
8. contournement des permissions ;
9. corruption de données.

12.2 Sécurité déjà réalisée

Des protections importantes existent déjà :

- autorisation backend ;
- authentification ;
- hashage des mots de passe ;
- permissions ;
- intégrité transactionnelle ;
- migrations contrôlées ;
- sauvegarde ;
- restauration ;
- backup pré-migration ;
- backup quotidien ;
- ACL ProgramData étudiées ;
- tests adversariaux ;
- tests de migration ;
- tests d'intégrité.

12.3 Backup/Restore V1

Recovery V1 a été accepté.

Fonctionnalités :

- backup manuel ;
- restore ;
- confirmation forte ;
- backup pré-migration ;

- backup quotidien local ;
- invalidation de session après restore.

Le système de backup utilise notamment :

```
.fmbak
SHA-256
```

pour la vérification d'intégrité dans l'état documenté.

12.4 Backup pré-migration

Avant migration :

```
DB
 ↓
backup
 ↓
migration
```

L'objectif est de ne jamais faire dépendre la récupération d'une base non sauvegardée.

12.5 Backup quotidien

Une sauvegarde locale automatique est prévue au maximum une fois par jour local.

12.6 ACL Windows

Une problématique réelle a été rencontrée :

```
SQLite Error 8
attempt to write a readonly database
```

La cause était liée aux ACL Windows du dossier ProgramData, et non au Dashboard.

Les permissions ont été corrigées de manière ciblée pour le compte de développement concerné.

Cela ne doit pas être transformé en :

```
Everyone = Full Control
```

sans justification.

12.7 Sécurité commerciale future

Un document de sécurité propose un futur chantier :

```
SECURITY-HARDENING-TRESORIA-01
```

avec notamment :

- licence signée cryptographiquement ;
- activation liée à l'installation/machine ;
- chiffrement SQLite ;
- chiffrement/authentification des `.fmbak` ;
- obfuscation .NET ;
- signature numérique EXE/installer ;
- anti-tamper ;
- sécurisation des updates.

Important : ces éléments ne doivent pas être considérés comme déjà implémentés.

12.8 Licence

L'idée étudiée est :

```
Installation
 ↓
Installation ID
 ↓
Machine fingerprint
 ↓
Licence signée
 ↓
Activation
```

La clé privée de signature doit rester hors de l'application.

L'application n'embarquerait que la clé publique nécessaire à la vérification.

12.9 Obfuscation

La documentation recommande :

```
Build Release
 ↓
Obfuscation
 ↓
Signature
 ↓
Installer
```

Mais l'obfuscation n'est pas considérée comme la sécurité principale.

Aucun secret critique ne doit être intégré dans le code.

12.10 Base chiffrée

Le chiffrement SQLite a été envisagé, par exemple avec une solution compatible SQLCipher.

Mais il faut résoudre en même temps :

```
Encryption
+
```

```
Backup
+
Disaster Recovery
```

afin de ne pas rendre une restauration sur un nouveau PC impossible.

12.11 Verrouillage manuel

Le besoin validé est :

```
Verrouiller
```

à côté de :

```
Se déconnecter
```

Car :

```
Responsable connecté
↓
quitte son bureau
↓
une autre personne arrive
```

L'application doit pouvoir être verrouillée volontairement.

Pas de verrouillage automatique par timeout.

Mot de passe du même utilisateur pour reprendre.

État final d'implémentation : à vérifier.

TRESORIA / PROJECT KNOWLEDGE BASE

13. EMAIL / CONTACT SYSTEM

Aucun système email/contact distant n'est documenté comme composant de Tresoria V1.

Le logiciel est local/offline.

Ne pas inventer :

- SMTP ;
- SendGrid ;
- Gmail API ;
- système de notifications email ;
- formulaire de contact distant.

TRESORIA / PROJECT KNOWLEDGE BASE

14. REVIEWS / RATINGS SYSTEM

Aucun système de :

- reviews ;
- ratings ;
- avis utilisateurs publics

n'a été récupéré comme fonctionnalité de Tresoria.

Cette section du modèle de backup générique ne correspond pas à une fonctionnalité connue de ce projet.

TRESORIA / PROJECT KNOWLEDGE BASE

15. ANALYTICS

Dashboard

Le Dashboard est **opérationnel**.

Il présente notamment :

- ventes ;
- achats ;
- dépenses ;
- résultat ;
- encaissements ;
- paiements fournisseurs ;
- trésorerie ;
- créances ;
- dettes ;
- alertes.

Reports

Les Rapports sont **analytiques**.

Ils permettent :

- synthèse ;
- comparaison ;
- évolution.

Règle importante

Les stocks financiers courants ne doivent pas être présentés comme des historiques lorsque la donnée historique n'existe pas.

Autrement dit :

> ne pas prétendre reconstruire artificiellement le stock de trésorerie historique.

TRESORIA / PROJECT KNOWLEDGE BASE

16. IMPORTANT UI/UX DETAILS

16.1 Design général

Le client a apprécié l'apparence générale du logiciel.

La direction retenue :

- moderne ;
- professionnel ;
- sobre ;
- lisible ;
- cohérent ;
- sans surcharge de couleurs.

16.2 Dashboard

Le Dashboard doit donner une impression immédiatement professionnelle.

Les graphes doivent être réels lorsque des données existent.

Les données à zéro doivent disposer d'un état vide propre plutôt qu'un graphique visuellement cassé.

16.3 Bug critique de Dashboard découvert

Après insertion de données réelles de démonstration, l'application pouvait se fermer après le login.

Cause :

un binding WPF `Run.Text` était par défaut en `TwoWay` alors que certaines propriétés étaient en lecture seule :

```
DueAlertItemViewModel.Reference
DueAlertItemViewModel.DueDate
```

Correction :

```
Mode=OneWay
```

Un test de régression a été ajouté.

Cette expérience est importante :

> une interface qui fonctionne avec une base vide ne doit jamais être considérée comme validée.

16.4 Bug des graphiques

Un problème avait été provoqué par :

```
InverseBooleanConverter
```

qui retournait un booléen alors que la cible était Visibility.

La correction a introduit une logique de visibilité explicite et mutuellement exclusive :

```
Chart visible
OU
Empty state visible
```

mais pas les deux.

16.5 Rapports

Les problèmes visuels corrigés comprenaient notamment :

- TabControl natif trop visible ;
- cadres imbriqués ;
- espaces déséquilibrés ;
- DataGrid de comparaison trop brut ;
- ComboBox ;
- focus ;
- scrollbars ;
- titres ;
- axes ;
- légendes ;
- empty states.

La validation finale de cette phase a été manuellement acceptée comme correcte sur les plans UI/UX.

16.6 Résultat

Convention visuelle :

```
positif → vert + gras
négatif → rouge + gras
zéro → neutre
```

Cette convention avait été demandée pour les rapports.

16.7 Clients

Dans la liste Clients :

```
Identifier
```

doit être masqué.

À la place :

```
Créance
```

```
```

doit être visible.

L'ID reste dans la base.

16.8 Fournisseurs

Dans Fournisseurs :

```
Identifier
```

doit être masqué.

À la place :

```
Dette
```

doit être affichée.

16.9 Paiements

La page principale doit être obligation-centric :

```
Vente
↓
Total
↓
Payé
↓
Reste
↓
Statut
```

Puis historique détaillé.

16.10 Reçu

Le reçu d'encaissement client cible :

```
80 mm thermal printer
```

avec environ :

```
72 mm content
4 mm margins
```

Contenu :

- EL BARKA EMBALLAGE ;
- logo ;
- date ;

- heure ;
- référence/ticket ;
- client ;
- référence vente ;
- total vente ;
- paiement courant ;
- total payé ;
- reste après ce paiement.

Règle historique critique

Le reçu historique doit afficher :

> le reste **après le paiement imprimé**

et non le reste actuel de la vente si d'autres paiements ont été effectués ensuite.

TRESORIA / PROJECT KNOWLEDGE BASE

17. DEPLOYMENT

17.1 Cible

Windows 10 principalement.

Windows 11 utilisé pour développement/UAT.

17.2 Packaging

Installateur prévu :

```
Inno Setup
```

Build :

```
Release
x64
self-contained
```

Nom envisagé :

```
GestionFinanciere_Setup_1.0.0.exe
```

Dans les documents historiques, une variante :

```
GestionFinanciere_Setup_1.0.0.exe
```

a également été évoquée.

Le nom final exact du fichier de release doit être confirmé lors du packaging final.

17.3 Installation

Application :

```
Program Files
```

Données :

```
C:\ProgramData\FinancialManagement
```

Backups :

```
C:\ProgramData\FinancialManagement\Backups
```

17.4 Tests de livraison prévus

Avant livraison :

```
Build Release
↓
Installer
↓
Installation propre
↓
Windows 10
↓
compte utilisateur standard
↓
login
↓
tests métier
↓
backup
↓
restore
↓
uninstall
↓
reinstall
```

TRESORIA / PROJECT KNOWLEDGE BASE

18. LOCAL DEVELOPMENT

DEV1

```
cd C:\Dev\logiciel-gestion-financiere
```

DEV2

```
cd C:\Users\INPT\Dev\logiciel-gestion-financiere
```

Principes

Avant toute implémentation :

1. définir précisément la tâche ;
2. définir les fichiers autorisés ;
3. définir les fichiers interdits ;
4. définir les règles métier ;
5. définir les tests ;
6. exécuter ;
7. vérifier ;
8. seulement après ACCEPT → Git.

Git

Règle importante :

> **Pas de add/commit/push pendant l'implémentation avant ACCEPT.**

Après ACCEPT :

```
opération Git explicite et séparée.
```

Pas de :

- force push ;
- reset improvisé ;
- rebase non demandé ;
- merge improvisé.

TRESORIA / PROJECT KNOWLEDGE BASE

19. BUGS AND PROBLEMS ENCOUNTERED

19.1 Dashboard navigation

Symptôme :

```
clic Dashboard
↓
Ventes reste affichée
```

Cause :

`DashboardViewModel` appelait :

```
InitializeCurrentMonth()
```

avant l'initialisation de :

```
_applyCustomRelayCommand
```

Cela provoquait un `NullReferenceException`.

Correction :

déplacer l'initialisation après la création de la commande.

Validation :

```
ViewModel navigation 6/6
Auth 15/15
Dashboard 31/31
Reports 6/6
Full 1107/1107
```

19.2 DueAlerts

Symptôme :

```
Login
↓
Dashboard
↓
application se ferme
```

Cause :

binding WPF incorrect sur propriété read-only.

Correction :

```
Mode=OneWay
```

Validation :

```
Full 1108/1108
```

19.3 SQLite readonly

Erreur :

```
SQLite Error 8
attempt to write a readonly database
```

Cause :

ACL Windows.

Pas une erreur de logique Dashboard.

19.4 NuGet timeout

Pendant une intégration :

```
SkiaSharp
OpenTK
```

n'étaient pas immédiatement récupérables à cause d'un problème de réseau/NuGet.

La revalidation ultérieure a réussi.

19.5 Backup invalid filename

Un fichier Daily mal nommé pouvait empêcher le mécanisme de retry.

Correction effectuée dans Recovery V1.

19.6 Backup startup service null

Une analyse Astra avait relevé qu'un service de backup optionnel pouvait théoriquement permettre un démarrage sans protection pré-migration.

Correction :

le service est devenu obligatoire.

19.7 Restore / migration

Une particularité du backup pré-migration autour du schéma 003 avait été identifiée :

une restauration d'un ancien artefact pouvait migrer immédiatement vers le schéma 004.

La décision a été acceptée car :

- l'ancien artefact restait intact ;

- les données étaient conservées ;
- la base atteignait le schéma actuel de manière sûre.

TRESORIA / PROJECT KNOWLEDGE BASE

20. COMPLETED FEATURES

Confirmés comme terminés/acceptés à différents checkpoints

Fondation

- architecture ;
- solution ;
- Domain/Application/Infrastructure/Desktop/Tests ;
- EF Core ;
- SQLite ;
- migrations initiales.

Auth

- login ;
- sessions ;
- hashage mots de passe ;
- permissions ;
- Responsable/Employé.

Ventes

- création ;
- clients connus ;
- ventes anonymes ;
- paiements.

Achats

- création ;
- fournisseur obligatoire ;
- paiements.

Trésorerie

- Cash ;
- Bank ;
- transferts ;
- soldes dérivés ;
- ouverture initiale.

Dépenses

- catégories ;
- dépense ;
- salaire ;
- obligation de renseigner l'employé concerné dans Recovery V1.

Clients/fournisseurs

- gestion de base.

Dashboard

- V2 ;

- périodes ;
- graphiques ;
- KPI ;
- alertes ;
- positions.

Reports

- Synthèse ;
- Comparaison ;
- Évolution.

Recovery V1

- Backup ;
- Restore ;
- pré-migration backup ;
- backup quotidien ;
- confirmation ;
- invalidation session après restore.

TRESORIA / PROJECT KNOWLEDGE BASE

21. PARTIALLY COMPLETED FEATURES

Cette section est particulièrement importante.

21.1 Modification des ventes

La modification complète n'est pas considérée comme définitivement terminée.

Problème :

```
Vente 3 000
↓
2 800 déjà payés
↓
montant devient 2 000
```

Il faut gérer :

- allocations ;
- avances ;
- cohérence ;
- trésorerie ;
- reste.

21.2 Modification des achats

Même problème côté fournisseur.

21.3 Modification des dépenses

Plus simple et avait été identifiée comme tâche restante.

État final exact à confirmer.

21.4 Suppression financière définitive

Capability conceptuelle présente, mais service complet de suppression contrôlée historiquement incomplet.

21.5 Export Excel

Demandé dans le cahier des charges, mais non confirmé comme terminé.

21.6 Backup/Restore avancé

Le V1 Recovery est accepté.

Mais les améliorations de sécurité suivantes restent distinctes :

- chiffrement ;
- authentification cryptographique renforcée ;
- gestion avancée WAL ;
- disaster recovery plus complet.

21.7 Reçu thermique

Le logiciel de reçu V1 a atteint :

```
1212 PASS
```

à son checkpoint.

Mais :

> **l'acceptation physique avec une vraie imprimante thermique n'était pas encore vérifiée.**

21.8 Verrouillage

Le besoin est clairement défini.

Mais l'état exact du dernier commit d'implémentation n'est pas suffisamment vérifié dans les informations récupérées ici.

TRESORIA / PROJECT KNOWLEDGE BASE

22. NOT IMPLEMENTED / FUTURE IDEAS

Sécurité commerciale

À envisager :

```
licence signée
machine binding
SQLite encryption
backup encryption
obfuscation
code signing
anti-tamper
update security
```

Fonctionnalités métier

Possibles futures versions :

- modifications avancées ventes ;
- modifications achats ;
- gestion avancée catégories ;
- avances autonomes ;
- plusieurs banques ;
- exports ;
- rapports plus avancés ;
- suppression contrôlée complète ;
- détails financiers plus poussés.

À ne pas ajouter automatiquement

Ne pas transformer Tresoria en :

- ERP ;
- comptabilité complète ;
- CRM ;
- SaaS ;
- système cloud complexe.

TRESORIA / PROJECT KNOWLEDGE BASE

23. CURRENT KNOWN ISSUES

À confirmer sur le HEAD final

Les points suivants ne doivent pas être déclarés « corrigés » sans nouvelle vérification :

1. état final de SEC-001 ;
2. état final du verrouillage ;
3. état final de l'installer ;
4. état final du code signing ;
5. état final du chiffrement SQLite ;
6. état final de la licence ;
7. état final des modifications Vente/Achat ;
8. état final de la suppression définitive ;
9. état final de l'export Excel ;
10. acceptation physique de l'imprimante thermique.

Problèmes reportés volontairement

Un audit sécurité avait explicitement classé comme reportés :

```
SEC-003 — Restore/WAL
SEC-004 — très grands montants
SEC-005 — mots de passe faibles
SEC-006 — base provenant d'une version future
SEC-007 — authenticité avancée des sauvegardes
```

ainsi que :

- chiffrement SQLite ;
- obfuscation ;
- licence complexe ;
- anti-tamper ;
- verrouillage automatique ;
- service Windows ;
- CI complexe.

TRESORIA / PROJECT KNOWLEDGE BASE

24. IMPORTANT DECISIONS

Décision 1

Monolithe modulaire.

Décision 2

SQLite local.

Décision 3

Pas de réseau inutile.

Décision 4

Montants en `long` / SQLite INTEGER.

Décision 5

Les soldes de trésorerie sont dérivés.

Décision 6

Les transferts ne changent pas le résultat.

Décision 7

Les paiements ne sont jamais automatiquement affectés.

Décision 8

Les avances sont dérivées des allocations.

Décision 9

Client anonyme autorisé mais obligatoirement payé intégralement.

Décision 10

Un employé peut voir les opérations des autres utilisateurs si son droit de lecture l'autorise.

Décision 11

Le Dashboard est opérationnel.

Décision 12

Reports est analytique.

Décision 13

Pas de preset “Cette semaine”.

Décision 14

Pas de verrouillage automatique par inactivité.

Décision 15

Verrouillage manuel distinct de Déconnexion.

Décision 16

Une sauvegarde doit exister avant une migration.

Décision 17

Backup quotidien local.

Décision 18

Pas de modification de schéma sans migration contrôlée.

TRESORIA / PROJECT KNOWLEDGE BASE

25. THINGS FUTURE AI/DEVELOPERS MUST NOT CHANGE

Cette section doit être considérée comme une **liste de garde-fous**.

Ne pas changer sans décision explicite

Argent

Ne jamais remplacer :

```
long / INTEGER
```

par :

```
double
float
```

pour les montants persistés.

Allocation

Ne jamais introduire silencieusement :

```
auto-allocation
```

des paiements.

Avances

Ne pas créer arbitrairement :

```
AdvanceBalance
```

comme état mutable.

Trésorerie

Ne pas stocker un :

```
CashBalance
BankBalance
```

mutable sans justification architecturale.

Transferts

Ne jamais compter un transfert interne comme :

```
Revenue
Expense
Result
```

Anonymous

Ne jamais permettre silencieusement :

```
Anonymous sale
+
unpaid remainder
```

Authorization

Ne jamais considérer :

```
bouton caché
```

comme une autorisation backend.

Database

Ne jamais créer une migration « rapide » pour résoudre un problème sans vérifier :

- Up ;
- Down ;
- données existantes ;
- contraintes ;
- drift.

Git

Ne jamais :

```
force push
reset --hard
rebase
```

sans décision explicite.

Migration ownership

Une seule personne/branche doit être propriétaire des migrations à un moment donné.

Scope

Ne pas ajouter une fonctionnalité importante simplement parce qu'elle semble intéressante.

TRESORIA / PROJECT KNOWLEDGE BASE

26. HOW TO REBUILD THE PROJECT FROM ZERO

Étape 1 — récupérer le repository

Utiliser le repository Git officiel.

Le nom exact du repository distant n'est pas récupéré de façon suffisamment fiable ici.

Étape 2 — récupérer la bonne branche

Ne pas supposer que :

```
release/v1.0-rc
```

est à jour.

À un checkpoint vérifié :

```
origin/integration
=
7650a487b36389cd0afdfaeae952bd6c9da969aa
```

tandis que :

```
origin/release/v1.0-rc
=
fea6f90da6b9c7e6ada3ef71d68dcc57a2edb4ba
```

Cela signifie que le RC était alors resté sur l'ancien baseline.

Étape 3 — installer .NET

Installer le SDK .NET 10 compatible.

Vérifier :

```
dotnet --version
```

Étape 4 — restaurer

Depuis le repository :

```
dotnet restore
```

Étape 5 — compiler

```
dotnet build
```

Étape 6 — tester

```
dotnet test
```

Le nombre attendu ne doit pas être inventé.

Il faut prendre le nombre du commit réellement checkouté.

Étape 7 — vérifier les migrations

Les migrations validées connues sont :

```
001
002
003
004
```

Étape 8 — base

Créer/utiliser :

```
C:\ProgramData\FinancialManagement\Data
```

avec :

```
FinancialManagement.db
```

Étape 9 — initialisation

Le workflow initial doit permettre :

- Responsable ;
- Cash opening balance ;
- Bank opening balance ;
- receivables initiales ;
- payables initiales.

Étape 10 — UAT

Tester :

```
login
vente
achat
paiement
créance
dette
dépense
transfert
Dashboard
Reports
backup
restore
permissions
```

TRESORIA / PROJECT KNOWLEDGE BASE

27. CURRENT PROJECT STATE SNAPSHOT

Baseline intégré vérifié

Le checkpoint d'intégration vérifié était :

```
origin/integration
7650a487b36389cd0afdfaeae952bd6c9da969aa
```

Il contenait :

- Recovery V1 ;
- Dashboard V2 ;
- Reports ;
- corrections UI finales correspondantes.

Release candidate vérifié à ce checkpoint

```
origin/release/v1.0-rc
fea6f90da6b9c7e6ada3ef71d68dcc57a2edb4ba
```

Il n'était pas encore mis à jour avec les deux grands chantiers ci-dessus.

Recovery V1

Commit :

```
1cb8d7cdb3bd838480237ca0138f63d5d5a80560
```

Merge integration :

```
aaa582336cba3dd23f1a6a430134aaaf9fdfd897
```

Dashboard/Reports

Feature commit :

```
360707f6cb8d5a6e4641cefd9880dc6d8d0a3f48
```

Integration merge :

```
7650a487b36389cd0afdfaeae952bd6c9da969aa
```

Payment work

Un checkpoint important :

```
7567a97c4a6c4e1f47b5007c775a9d236b0cf2e0
```

a porté l'UX obligation-centric.

Receipt

Checkpoint :

```
4baadfe178b332befd0be4c5a8976184f32dd901
```

et :

```
1212 PASS
```

au checkpoint Receipt V1.

Mais la convergence finale de ce travail avec le HEAD actuel n'est pas vérifiée ici.

TRESORIA / PROJECT KNOWLEDGE BASE

28. NEXT DEVELOPMENT STEPS

Priorité 1 — établir le vrai HEAD

Avant toute nouvelle modification :

```
git status
git branch
git log
git remote
```

Puis comparer :

```
feature branches
integration
release/v1.0-rc
origin/*
```

Priorité 2 — vérifier les fonctionnalités post-integration

En particulier :

- paiements ;
- Clients ;
- Fournisseurs ;
- reçu ;
- verrouillage ;
- sécurité session.

Priorité 3 — finaliser UAT

Tester avec données réalistes.

Priorité 4 — installer

Construire :

```
Release x64 self-contained
```

puis :

```
Inno Setup
```

Priorité 5 — installation réelle

Tester sur :

```
Windows 10
```

avec utilisateur non administrateur.

Priorité 6 — disaster recovery

Faire :

```
DB
↓
backup
↓
problème DB
↓
restore
↓
restart
↓
vérification
```

Priorité 7 — final security review

Astra doit vérifier :

- auth ;
- authorization ;
- session ;
- ProgramData ;
- backup ;
- restore ;
- migrations ;
- corruption ;
- permissions ;
- installer.

TRESORIA / PROJECT KNOWLEDGE BASE

29. RECOVERY CHECKLIST

Avant suppression des anciens chats/fichiers

Git

- [ ] repository local sauvegardé
- [ ] remote vérifié
- [ ] branches importantes sauvegardées
- [ ] integration sauvegardée
- [ ] release sauvegardée
- [ ] branches payment sauvegardées
- [ ] receipt sauvegardée
- [ ] lock sauvegardée
- [ ] tous les commits importants conservés

Documents

Conserver :

- [ ] cahier des charges ;
- [ ] modélisation fonctionnelle ;
- [ ] modèle de données ;
- [ ] UX/UI ;
- [ ] architecture ;
- [ ] audits Astra ;
- [ ] rapports Recovery ;
- [ ] rapports Dashboard ;
- [ ] rapports Payment ;
- [ ] rapports Receipt ;
- [ ] fixture manifest ;
- [ ] documents sécurité.

Données de test

Conserver si utile :

```
fixture-manifest.committed.json
```

et documenter que les données étaient des données de démonstration/UAT.

Environnement

Conserver hors du chat :

- scripts d'installation ;
- scripts de test ;
- scripts de seed ;

- fichiers Inno Setup ;
- configuration de build ;
- documentation locale ;
- procédures de backup/restore.

TRESORIA / PROJECT KNOWLEDGE BASE

30. INFORMATION THAT COULD NOT BE RECOVERED

Cette section est volontairement explicite.

Non récupéré avec certitude

1. Nom exact du repository Git distant

Pas suffisamment documenté dans les informations disponibles.

2. HEAD Git réellement actuel

Le dernier état **certain** est l'intégration :

```
7650a487...
```

Des développements ultérieurs ont existé.

Leur convergence finale n'est pas vérifiée.

3. État final du `release/v1.0-rc`

Le dernier état certain est :

```
fea6f90...
```

Ne pas supposer qu'il a depuis été mis à jour.

4. Nom exact de toutes les variables d'environnement

Non récupéré.

5. Secrets

Aucun secret ne doit être reconstitué dans ce document.

6. Certificat de code signing

Non récupéré.

7. Clé privée de licence

Non récupérée — et elle ne doit **jamais** être stockée dans ce document ou dans le repository.

8. État final du verrouillage

Besoin confirmé, état final non suffisamment vérifié.

9. État final de l'installer

Le workflow est connu mais l'artefact final exact n'est pas confirmé.

10. Test physique imprimante

Pas confirmé.

11. Convergence finale Payment/Receipt/Lock

Non confirmée.

TRESORIA / PROJECT KNOWLEDGE BASE

31. FILES I MUST KEEP OUTSIDE CHATGPT

C'est probablement la section la plus importante pour ton objectif de suppression des anciennes conversations.

À sauvegarder impérativement

A. Repository Git complet

Conserver le repository **avec son `.git`**.

Ne pas conserver seulement le dossier source sans historique.

B. Toutes les branches importantes

Au minimum les branches correspondant aux travaux :

```
integration
release/v1.0-rc
feature/release-recovery-v1
feature/dashboard-v2
payment-related branches
receipt-related branch
lock-related branch
```

Les noms exacts des dernières branches payment/lock doivent être vérifiés avec Git.

C. Documents de conception

Conserver :

```
Cahier des charges fonctionnel V1.0
Modélisation fonctionnelle complète V1.0
Conception modèle de données V1.0
Conception UX/UI V1.0
Architecture Technique V1.0
```

D. Audits

Conserver :

```
UAT-V1-COMPLETENESS-AUDIT.md

DASHBOARD-V2-STARTUP-DIAGNOSIS.md
DASHBOARD-V2-RUNTIME-CRASH-REPORT.md
DASHBOARD-V2-FINAL-VISUAL-QA-REPORT.md
DASHBOARD-V2-REPORT.md
```

et tous les rapports Astra/Recovery/Payment/Receipt.

E. Scripts

Conserver tous les scripts :

- migration ;
- test ;
- seed ;
- UAT ;
- installation ;
- packaging ;
- backup/restore.

F. Installer

Une fois créé :

```
GestionFinanciere_Setup_1.0.0.exe
```

ou son nom final exact.

G. Fixture

Conserver :

```
fixture-manifest.committed.json
```

si les données de démonstration doivent être reproduites.

H. Informations de licence

Ne jamais stocker les clés privées dans le chat.

Conserver les secrets dans un coffre externe sécurisé.

TRESORIA / PROJECT KNOWLEDGE BASE

32. MASTER RESUME PROMPT

Voici le prompt à conserver. Il est conçu pour permettre à une nouvelle conversation/IA de reprendre le projet.

MASTER RESUME PROMPT — TRESORIA

PROJECT: TRESORIA — LOGICIEL DE GESTION FINANCIÈRE

CLIENT:
EL-BARKA EMBALLAGE

TYPE:
Application desktop Windows locale/offline de gestion financière opérationnelle.

STACK:
C#
.NET 10 LTS
WPF
MVVM
Entity Framework Core
SQLite
xUnit
LiveChartsCore.SkiaSharpView.WPF 2.0.5
Inno Setup

ARCHITECTURE:
Monolithe modulaire.

Projects:
FinancialManagement.Domain
FinancialManagement.Application
FinancialManagement.Infrastructure
FinancialManagement.Desktop
FinancialManagement.Tests

DATA:
SQLite local.
Production DB:
C:\ProgramData\FinancialManagement\Data\FinancialManagement.db

BACKUPS:
C:\ProgramData\FinancialManagement\Backups

MONEY:
Domain = long.
SQLite = INTEGER.
Never use float/double for persisted financial amounts.

VALIDATED MIGRATIONS:
20260903232954_001_InitialSchema
20260904004853_002_AllowAnonymousSalePayments
20260904111121_003_TreasuryOpeningBalanceSingleton
20260908072131_004_EnforceFinancialIntegerStorageClass

NO MIGRATION 005 IS PART OF THE KNOWN VALIDATED BASELINE.

CORE BUSINESS RULES:

Sales:
- automatic date/reference
- optional known client
- anonymous sales allowed
- anonymous sales must be fully paid
- optional observation
- payment does not count as a second sale/result

Purchases:
- supplier mandatory
- automatic date/reference
- optional observation
- payment immediate or deferred

Payments:
- Cash
- BankTransfer
- multiple payments allowed
- partial payments allowed
- payments can be allocated to one or several obligations
- allocation is explicit
- never auto-allocate silently
- overpayment becomes derived advance
- no mutable AdvanceBalance table

Receivables/payables:
- derived from obligations minus allocations
- initial receivables/payables supported

Treasury:
- one Cash
- one Bank
- balances derived
- negative balances allowed
- internal transfers do not affect result

Result:
Sales - Purchases - Expenses

Expenses:
- categories:
  Salaires
  Transport
  Loyer
  Factures
  Divers
- salary expense requires concerned employee
- category already used must not be arbitrarily deleted/changed

USERS:
RESPONSABLE
EMPLOYÉ

Employee permissions:
- Ventes
- Achats
- Dépenses

- Clients
- Fournisseurs
- Encaissements clients
- Paiements fournisseurs

Actions:
- Voir
- Créer
- Modifier

Important:
Employee cross-user READ visibility is intentionally accepted.
Do not remove it.
Employee modification of other users' operations must not become unrestricted.

RESPONSABLE:
Responsible-only functions include global Dashboard, Treasury, Administration, Backup/Restore, Reports and sensitive administrative operations according to actual implementation.

UI:
Modern, professional, restrained.
Do not add gratuitous colors.
Dashboard is operational.
Reports is analytical.

DASHBOARD PERIODS:
Aujourd'hui
Ce mois
Cette année
Personnalisé

NO "Cette semaine".

Dashboard graph granularity:
Today = one daily point
Month = daily
Year = monthly Jan-Dec
Custom <=62 days = daily
Custom >62 days = monthly

REPORTS:
Synthèse
Comparaison
Évolution

Comparison labels:
Mois analysé
Mois de comparaison
Comparer

RESULT VISUAL:
positive = green + bold
negative = red + bold
zero = neutral

CLIENTS:
Hide Identifier in visible list.
Show current Créance.
Keep ID in database.

SUPPLIERS:

Hide Identifier in visible list.
Show current Dette.
Keep ID in database.

PAYMENT UX:
Main list is obligation-centric.
Customer collection:
1 row = 1 sale/receivable.
Show:
client
sale reference
initial amount
total paid
remaining
status
actions

Details:
show chronological individual payments and remaining after each payment.

Supplier payment page follows the same purchase/debt-centric model.

RECEIPT:
Customer collection receipt.
Target thermal printer = 80 mm.
Known target content:
EL BARKA EMBALLAGE
logo
date/time
ticket/reference
client
sale reference
sale total
current payment
total paid
remaining after this payment

Critical historical rule:
Receipt must show remaining AFTER THAT PAYMENT, not the current later remaining balance.

BACKUP/RECOVERY:
Recovery V1 was accepted.
Includes:
manual backup
restore
strong confirmation
pre-migration backup
daily local backup at most once per local business date
session invalidation after restore

.fmbak exists.
SHA-256 integrity verification exists in the documented V1 state.

SECURITY:
Backend authorization is mandatory.
Hidden UI button is NOT a security boundary.
Use:
UI -> ViewModel -> Application Service -> Authorization -> Business Rule -> DB

Known security future hardening:
SECURITY-HARDENING-TRESORIA-01

Potential future:
signed license
machine-bound activation
SQLite encryption
encrypted/authenticated backups
.NET obfuscation
EXE/installer signing
anti-tamper
secure updates

Do NOT assume these future protections are already implemented.

MANUAL LOCK:
Required behavior:
- "Verrouiller" beside "Se déconnecter"
- manual only
- no automatic inactivity timeout
- hide application
- same user password required to unlock
- available to Responsable and Employé
Final implementation status must be verified from Git before claiming complete.

SECURITY FINDINGS:
Some security findings were intentionally deferred:
SEC-003 Restore/WAL
SEC-004 extremely large monetary amount
SEC-005 weak passwords
SEC-006 future-schema DB
SEC-007 advanced backup authenticity

Do not reopen these automatically unless release scope requires it.

KNOWN TECHNICAL BUGS THAT WERE FIXED:
1. Dashboard navigation NullReference:
DashboardViewModel initialized current month before command creation.
Fixed by moving initialization after command creation.

2. DueAlerts WPF runtime crash:
Run.Text default TwoWay binding targeted read-only Reference/DueDate.
Fixed with Mode=OneWay.

3. Dashboard chart visibility:
InverseBooleanConverter returned bool for Visibility.
Fixed with target-aware visibility and mutually exclusive chart/empty state.

4. SQLite readonly:
Caused by Windows ACL on ProgramData, not Dashboard.
Environment issue.

IMPORTANT GIT CHECKPOINTS:
Frozen baseline:
fea6f90da6b9c7e6ada3ef71d68dcc57a2edb4ba

Recovery V1 accepted commit:
1cb8d7cdb3bd838480237ca0138f63d5d5a80560

Recovery integration merge:
aaa582336cba3dd23f1a6a430134aaaf9fdfd897

Dashboard/Reports feature:
360707f6cb8d5a6e4641cefd9880dc6d8d0a3f48

Dashboard/Reports integration:
7650a487b36389cd0afdfaeae952bd6c9da969aa

Payment obligation-centric checkpoint:
7567a97c4a6c4e1f47b5007c775a9d236b0cf2e0

Receipt branch checkpoint:
4baadfe178b332befd0be4c5a8976184f32dd901

IMPORTANT:
The final convergence of later payment/receipt/lock work was NOT verified in the backup context.
Do NOT assume these commits are in origin/integration.
Check Git first.

LAST VERIFIED INTEGRATION:
origin/integration =
7650a487b36389cd0afdfaeae952bd6c9da969aa

LAST VERIFIED RELEASE RC AT THAT CHECKPOINT:
origin/release/v1.0-rc =
fea6f90da6b9c7e6ada3ef71d68dcc57a2edb4ba

DEVELOPMENT WORKFLOW:

Human lead controls:
- business rules
- scope
- architecture
- acceptance
- release decisions

AI/Codex executes bounded tasks.

Model strategy:
5.6 Luna:
scoped Git/prep/UI/freeze/merge/push/UAT support

5.6 Sol:
substantive implementation/features/integration/logic

6 Astra:
architecture/security/migrations/adversarial audits/hard bugs/release review

RULE:
Never accept AI output merely because tests exist.
Review:
- implementation
- hidden cases
- migrations
- invariants
- authorization
- persistence
- runtime
- UI
- security

GIT RULE:
No add/commit/push during implementation before ACCEPT.
After ACCEPT, Git operation is a separate explicit step.

Never improvise reset/rebase/force push.

DEV SPLIT:

DEV1:
C:\Dev\logiciel-gestion-financiere
Owns:
architecture
backend
database
security
financial core
migrations
backup/recovery

DEV2:
C:\Users\INPT\Dev\logiciel-gestion-financiere
Owns:
WPF
MVVM
Shell
navigation
UI
Dashboard
simple integration

One migration owner at a time.

CURRENT RECOVERY WARNING:
Before new work, first inspect:
git status
git branch -a
git log
git remote -v
git rev-parse HEAD
git rev-parse origin/integration
git rev-parse origin/release/v1.0-rc

Then determine the true current HEAD.

DO NOT assume the historical integration SHA is still current.

FINAL DELIVERY:
Release x64 self-contained.
Inno Setup.
Install Program Files.
Data in ProgramData.
Test Windows 10 with standard user.
Test:
installation
login
sales
purchases
payments
receivables
payables
expenses
treasury
Dashboard
Reports
permissions

backup
restore
uninstall
reinstall
restart

IMPORTANT DOCUMENTS:
Comprendre le métier client.txt
Sécuriser l’application installée.txt
Cahier des charges fonctionnel V1.0 validated
Modélisation fonctionnelle complète V1.0 validated
Conception modèle de données V1.0 validated
Conception UX/UI V1.0 validated
Architecture Technique V1.0 validated
UAT-V1-COMPLETENESS-AUDIT.md
DASHBOARD-V2-STARTUP-DIAGNOSIS.md
DASHBOARD-V2-RUNTIME-CRASH-REPORT.md
DASHBOARD-V2-FINAL-VISUAL-QA-REPORT.md
DASHBOARD-V2-REPORT.md
fixture-manifest.committed.json
Recovery/Astra/Payment/Receipt reports

RESTART RULE:
If information conflicts, do NOT silently choose.
Identify:
1. verified current Git state
2. historical decision
3. later change
4. unresolved discrepancy

Then ask/verify before modifying the project.

MOST IMPORTANT PRINCIPLE:
Tresoria is a financial application.
Data integrity > feature count.
Correctness > speed.
Security > convenience.
Simple reliable V1 > over-engineered V1.

TRESORIA / PROJECT KNOWLEDGE BASE

APPENDIX A — ENGINEERING WORKFLOW RECOVERED

Le projet n'a pas suivi un simple :

```
idea → code → release
```

Le processus réellement utilisé est beaucoup plus proche de :

```
Discovery
  ↓
Requirements
  ↓
Business rules
  ↓
CDC
  ↓
Functional model
  ↓
Data model
  ↓
UX/UI
  ↓
Architecture
  ↓
Implementation
  ↓
Targeted tests
  ↓
Full tests
  ↓
Build
  ↓
Migration verification
  ↓
Security audit
  ↓
Runtime UAT
  ↓
Visual UAT
  ↓
ACCEPT
  ↓
Git freeze
  ↓
Controlled integration
  ↓
Regression
  ↓
Release
```

C'est un des éléments les plus importants à préserver.

TRESORIA / PROJECT KNOWLEDGE BASE

APPENDIX B — QUALITY GATES

Les gates de conception explicitement documentés comprennent :

```
MODEL-SALES-01          PASS
PURCHASE                PASS
EXPENSE                 PASS
TREASURY                PASS
PARTIES                 PASS
USERS                   PASS
INITIALIZATION          PASS
ALERTS                  PASS
DASHBOARD               PASS
FUNCTIONAL-MODEL-V1     PASS
DATA-MODEL-V1           PASS
UX-UI-V1                PASS
ARCHITECTURE-TECHNIQUE-V1  PASS
```

TRESORIA / PROJECT KNOWLEDGE BASE

APPENDIX C — UAT DATASET KNOWN

Un dataset temporaire de démonstration a été utilisé pour visualiser le Dashboard.

Il contenait notamment :

```
5 clients
5 fournisseurs
36 ventes
27 achats
18 dépenses
39 paiements clients
28 paiements fournisseurs
```

État financier de cette fixture :

```
Créances clients : 42 620
Dettes fournisseurs : 41 740

Cash : 26 830
Bank : 67 120
Treasury : 93 950
```

Pour septembre :

```
Sales      19 900
Purchases  14 300
Expenses    1 850
Result      3 750
```

Ces données étaient destinées à la visualisation/UAT et ne doivent pas être confondues avec les données métier réelles du client.

TRESORIA / PROJECT KNOWLEDGE BASE

APPENDIX D — FINAL HANDOFF RULE

Si ce document est utilisé après suppression des conversations précédentes, **ne pas commencer directement à coder**.

La première opération doit être :

1. récupérer le repository
2. vérifier Git
3. vérifier les branches
4. identifier le vrai HEAD
5. comparer avec 7650a487...
6. identifier les commits Payment/Receipt/Lock présents ou absents
7. lancer les tests
8. vérifier le build
9. seulement ensuite reprendre le développement

Et surtout :

> **Ce document constitue une mémoire de projet, pas une preuve que tous les développements postérieurs au checkpoint `7650a487...` sont intégrés. La vérité finale du code doit toujours être déterminée par le repository Git et les tests du HEAD actuel.**

État de confiance du présent handoff

Élément	État
Métier V1	**Très bien documenté**
Architecture	**Vérifiée/documentée**
Stack	**Vérifiée**
Modèle financier	**Vérifié**
Migrations 001–004	**Vérifiées**
Recovery V1	**Accepté**
Dashboard/Reports	**Accepté**
UI Dashboard/Reports	**Acceptée manuellement**
Payment UX	**Implémentée à un checkpoint, convergence finale à vérifier**
Receipt	**Implémenté à un checkpoint, impression physique à vérifier**
Lock	**Besoin confirmé, état final à vérifier**
Installer final	**À vérifier**
Release finale V1.0	**À vérifier**
HEAD Git actuel	**À vérifier**
Secrets	**Intentionnellement non récupérés**